GUIDE TECHNIQUE D'IMPLÉMENTATION · V1.0

Adaptation de ctrader-openapi-rust à un moteur d'arbitrage Rust

Spécifications protocolaires, architecture runtime et code de production pour IC Markets cTrader Open API v2. Document de référence pour l'adaptation au moteur utilustrader.io.

89	15	LD5
PAYLOADTYPES COUVERTS (2100–2188)	CHAPITRES TECHNIQUES	CIBLE VPS EQUINIX LONDON

RUST · TOKIO · RUSTLS | PROTOBUF V2 | IC MARKETS | ARBITRAGE TRIANGULAIRE | HFT

Plateforme cible · utilustrader.io
Dépôt source · github.com/raul-gherman/ctrader-openapi-rust

Version · 1.0
Date · 2026-11 · Europe/Paris

Table des matières

Contexte et périmètre 4

Corrections importantes par rapport aux croyances courantes . . . 4
Cible opérationnelle . . . 4
Cartographie du document . . . 5

Architecture du protocole cTrader Open API 5

Endpoint IC Markets . . . 5
L'enveloppe ProtoMessage . . . 5
Le piège du frame header 4 octets . . . 6
Règle du Heartbeat (maintien de connexion) . . . 6
Workflow d'exécution d'une session complète . . . 6

Cartographie des types générés dans spotware.rs 7

Énumérations principales . . . 7
PayloadType essentiels pour l'arbitrage . . . 7
Structures de messages clés . . . 8

Configuration du projet Cargo et dépendances 8

Fichier Cargo.toml . . . 9
Arborescence cible du crate . . . 10
Choix techniques et justifications . . . 10
Commande de build release . . . 11

Connexion TCP/TLS et codec de tramage 11

Structure Connection . . . 11
Codec de tramage : subtilités et pièges . . . 13
Test de round-trip et errors protocolaires . . . 13

Authentification à deux niveaux 14

Étape 1 — Application Auth (2100 / 2101) . . . 14
Étape 2 — Account Auth (2102 / 2103) . . . 15
Pattern de matching par clientMsgId . . . 15
Gestion du token expiry . . . 16

Maintien de connexion : heartbeat et watchdog 16

Tâche heartbeat d'envoi . . . 17

Watchdog côté réception . . . 17
Intégration dans le dispatch loop . . . 18
Récupération des symboles et abonnement aux spots 19
Étape 1 — Récupération de la liste des symboles . . . 19
Étape 2 — Abonnement aux spots (2127) . . . 20
SpotManager : structure thread-safe . . . 20
Conversion int64 → f64 et gestion des décimales . . . 21
Envoi d'ordres et suivi d'exécution 21
Champs de ProtoOANewOrderReq . . . 22
OrderManager : envoi et suivi . . . 23
Gestion des ProtoOAOrderErrorEvent (2132) . . . 24
Logique d'arbitrage triangulaire EUR → USD → CHF → EUR 25
Formule mathématique . . . 25
Tableau de décision BUY/SELL par jambe . . . 25
Implémentation du détecteur d'arbitrage . . . 26
Exécution simultanée et compensation 27
Exécution parallèle via tokio::join! . . . 28
Suivi des positions via ProtoOAReconcileReq . . . 29
Optimisations HFT : VPS LD5, lock-free, pinning CPU 29
Colocalisation VPS LD5 IBX Equinix . . . 29
Tuning du runtime Tokio . . . 30
Files lock-free : crossbeam-queue . . . 30
Zero-copy avec bytes::Bytes . . . 31
Désactivation de Nagle et tuning socket . . . 31
Mesure de latence : hdrhist . . . 31
Gestion d'erreurs et résilience 32
Reconnexion avec backoff exponentiel . . . 33
Mapping ProtoOAErrorCode → actions . . . 33
Slippage guard . . . 34
Logging structuré avec rotation . . . 35
Mode démo, backtesting et replay de ticks 35
Paper Trading Interceptor . . . 35

Replay Engine : rejouer des ticks enregistrés . . . 36

Métriques de backtesting . . . 37

Checklist de mise en production **38**

Sécurité et configuration . . . 38

Test et validation . . . 38

Monitoring et alerting . . . 39

Risque et capital . . . 39

Référence des PayloadType (2100–2188) **39**

Références et ressources externes . . . 41

CHAPITRE 1 · INTRODUCTION

Contexte et périmètre

Le dépôt GitHub **raul-gherman/ctrader-openapi-rust** fournit une base de code Rust préalable à toute implémentation d'un client cTrader : il contient un fichier unique, `spotware.rs`, de plus de 6 500 lignes, généré automatiquement depuis les fichiers *.proto* de Spotware à l'aide de la chaîne d'outils **quack-builder** et de la crate **quacky**. Ce fichier expose l'intégralité des types protobuf de l'Open API v2 — énumérations de PayloadType, structures de requêtes et de réponses, codes d'erreur — mais ne fournit **aucune logique réseau, aucun handshake TLS, aucune gestion de heartbeat, aucune authentification**. Toute cette couche runtime reste à construire.

Le présent document comble ce vide. Il décrit précisément comment adapter ce dépôt à un moteur d'arbitrage triangulaire opérationnel sur IC Markets, en s'appuyant sur le runtime asynchrone **Tokio**, le moteur TLS **rustls** (sans dépendance OpenSSL, indispensable pour cross-compilation vers VPS ARM), et les patterns d'architecture HFT les plus éprouvés : files *lock-free*, *CPU pinning*, zéro copie mémoire.

Corrections importantes par rapport aux croyances courantes

Avant d'entrer dans l'implémentation, il est indispensable de corriger plusieurs erreurs répandues dans la documentation communautaire cTrader, qui conduisent systématiquement à des déconnexions immédiates ou à des comportements erratiques :

Croyance courante	Réalité (vérifiée dans spotware.rs)	Conséquence si erroné
Heartbeat = payloadType 2104	2104 = PROTO_OA_VERSION_REQ ; Heartbeat = 51 (HEARTBEAT_EVENT)	Le serveur renvoie un ERROR_RES (50) et coupe la socket
ProtoOANewOrderReq = 2128	2128 = PROTO_OA_SUBSCRIBE_SPOTS_RES ; ordre = 2106	Aucun ordre ne partira jamais
Heartbeat obligatoire toutes les 30s côté client	Le serveur envoie 51 toutes les ~30s ; client recommandé à 10s	Risque de timeout silencieux si watchdog mal codé
Port 5034 pour demo	5035 pour démo ET réel ; 5034 historiquement pour PROFIT	Connexion impossible sur le mauvais port

Tableau 1 — Pièges protocolaires les plus fréquents

Cible opérationnelle

La cible est un moteur d'arbitrage triangulaire EUR → USD → CHF → EUR fonctionnant sur IC Markets, broker ECN offrant un accès cTrader avec un *freeze level* à 0, des ordres autorisés dans le spread, et un *matching engine* hébergé au datacenter Equinix LD5 (London, Slough). Le déploiement en production doit se faire sur un VPS colocalisé LD5 pour minimiser la latence aller-retour (objectif : < 2 ms), et le moteur doit supporter un débit de 2 000 ticks par seconde sur les trois paires simultanément, sans bloquer la *event loop* Tokio.

Cartographie du document

Les chapitres 2 à 4 posent les fondations protocolaires et le setup Cargo. Les chapitres 5 à 9 implémentent la *stack* réseau complète (TCP/TLS, codec, auth, heartbeat, abonnement spots, envoi d'ordres). Les chapitres 10 et 11 construisent la logique métier d'arbitrage. Les chapitres 12 à 14 traitent des optimisations HFT, de la résilience et du backtesting. Le chapitre 15 est une checklist de mise en production. L'annexe A contient la table exhaustive des PayloadType (2100 à 2188).

CHAPITRE 2 · PROTOCOLE

Architecture du protocole cTrader Open API

L'Open API v2 de Spotware n'est pas une API REST : c'est une **connexion TCP persistante chiffrée en TLS 1.2+** sur laquelle transitent des messages binaires sérialisés en Protocol Buffers. Il n'y a ni requête-réponse HTTP, ni pagination, ni état implicite côté serveur en dehors de la socket. Tout est piloté par le client Rust via l'envoi et la réception asynchrones de *frames* binaires.

Endpoint IC Markets

Hôte	Port	TLS	Usage
ctraderapi.com	5035	TLS 1.2+	Démo et réel : authentification, ticks, ordres
ctraderapi.com	5034	TLS 1.2+	Historique (PROFIT) — non requis pour l'arbitrage live

Tableau 2 — Endpoints IC Markets cTrader Open API

L'enveloppe ProtoMessage

Chaque message échangé est encapsulé dans une enveloppe unique définie dans `spotware.rs` sous le nom `ProtoMessage`. Trois champs composent cette enveloppe :

Champ	Type Protobuf	Rôle
payloadType	int32 (requis)	Identifie le type de message (ex: 2100 = app auth req). Table de routage universelle.
payload	bytes (optionnel)	Le message réel sérialisé en protobuf. Vide pour le Heartbeat.
clientMsgId	string (optionnel)	Corrélation requête/réponse : le serveur renvoie ce même ID dans la résponse.

Le champ **clientMsgId** est critique pour un moteur d'arbitrage : il permet d'associer sans ambiguïté chaque `ProtoOAExecutionEvent` (2126) reçu à l'ordre spécifique qui l'a déclenché, et donc de résoudre la *Future* Rust correspondante via un canal *oneshot*. Sans lui, en cas de trois ordres simultanés (jambes d'arbitrage), il devient impossible de savoir quel ordre a été rempli.

Le piège du frame header 4 octets

> **CRITIQUE · FRAME HEADER BIG-ENDIAN**
> Avant chaque `ProtoMessage` sérialisé envoyé sur le socket, votre moteur DOIT écrire un **uint32 en Big-Endian** (4 octets) contenant la taille exacte en octets du message Protobuf qui suit. Si la taille est fausse, ou si l'en-tête est omis, le serveur IC Markets clôt **immédiatement** la connexion sans message d'erreur — vous recevez au mieux un `ERROR_RES` (50) avec code `FRAME_TOO_LONG` (8) avant la coupure.

Côté réception, la logique est symétrique : lire 4 octets, interpréter en Big-Endian pour obtenir **N**, puis lire exactement **N** octets pour reconstituer le `ProtoMessage`. La structure binaire d'une trame est donc :

ASCII · FRAME LAYOUT

```
┌──────────────┬──────────────────────────────────────────┐
│  4 octets BE │  N octets : ProtoMessage sérialisé       │
│  = N (uint32)│  (payloadType + payload + clientMsgId)   │
└──────────────┴──────────────────────────────────────────┘

               ↑
               Pas de marqueur de début, pas de marqueur de fin.
               Tout repose sur la connaissance exacte de N.
```

Figure 1 — Format de trame cTrader sur le socket TLS

Règle du Heartbeat (maintien de connexion)

Le serveur cTrader envoie spontanément un `ProtoHeartbeatEvent` (payloadType = 51, payload vide) toutes les **30 secondes environ**. Ce heartbeat server-to-client prouve que la connexion est vivante. La documentation officielle Spotware recommande également au client d'envoyer son propre heartbeat toutes les 10 secondes, bien que ce ne soit pas strictement obligatoire :

- **Côté serveur → client** : ProtoHeartbeatEvent (51) toutes les ~30s. Si absent pendant > 35s, considérer la connexion morte et lancer une reconnexion.
- **Côté client → serveur** : ProtoHeartbeatEvent (51) toutes les 10s. Recommandé mais non bloquant en pratique. À implémenter systématiquement en production.

Un point subtil : **il n'y a pas d'acquittement** des heartbeats. Le serveur ne renvoie pas de réponse spécifique à votre heartbeat — il se contente de maintenir la connexion ouverte. Le seul moyen de détecter une connexion morte est l'absence de heartbeat serveur pendant une durée supérieure à 35-40 secondes.

Workflow d'exécution d'une session complète

Une session d'arbitrage typique suit cet enchaînement strict. Chaque étape est bloquante : il faut attendre la résponse de l'étape N avant d'envoyer la requête de l'étape N+1, sous peine de recevoir un `CH_CLIENT_NOT_AUTHENTICATED` (102) ou `ACCOUNT_NOT_AUTHORIZED` (2).

WORKFLOW · SESSION ARBITRAGE

```
[1] TCP connect → ctraderapi.com:5035  (avec TCP_NODELAY)
[2] TLS handshake (SNI = ctraderapi.com)
[3] ProtoOAApplicationAuthReq   (2100) → attente 2101
[4] ProtoOAAccountAuthReq       (2102) → attente 2103
[5] ProtoOASymbolsListReq       (2114) → attente 2115  (récup. symbolId numériques)
[6] ProtoOASubscribeSpotsReq    (2127) → attente 2128  (pour chaque symbole)
[7] Boucle de réception asynchrone des ProtoOASpotEvent (2131) ← bid/ask en continu
[8] Détection d'opportunité d'arbitrage → ProtoOANewOrderReq (2106) × 3 en parallèle
```

```
[9] Réception ProtoOAExecutionEvent (2126) × 3 → résolution via clientMsgId
[10] Heartbeat (51) toutes les 10s en parallèle
```

Figure 2 — Séquence protocolaire complète d'une session d'arbitrage

CHAPITRE 3 · TYPES PROTOBUF

Cartographie des types générés dans spotware.rs

Le fichier `spotware.rs` du dépôt raul-gherman/ctrader-openapi-rust expose les types Rust suivants, générés depuis `spotware.95.proto`. Tous dérivent de `quacky::MessageRead` et `MessageWrite`, ce qui fournit les méthodes `.read()` et `.write()` pour la sérialisation binaire. Aucune dépendance externe à *prost* ou *protobuf* n'est requise.

Énumérations principales

Le dépôt définit trois énumérations de routing, organisées hiérarchiquement. La première (`ProtoPayloadType`) couvre les messages de niveau transport ; la deuxième (`ProtoOAPayloadType`) couvre tous les messages métier de l'Open API ; la troisième regroupe les codes d'erreur.

Énumération	Range	Nombre	Usage
ProtoPayloadType	5 / 50 / 51	3	Transport : PROTO_MESSAGE, ERROR_RES, HEARTBEAT_EVENT
ProtoOAPayloadType	2100 → 2188	89	Tous les messages métier Open API (req/res/events)
ProtoErrorCode	1 → 11	10	Erreurs transport (FRAME_TOO_LONG, TIMEOUT_ERROR...)
ProtoOAErrorCode	1 → 136	~50	Erreurs métier (NOT_ENOUGH_MONEY, SYMBOL_NOT_FOUND...)
ProtoOADayOfWeek	0 → 7	8	Jours de semaine (sessions, swaps)
ProtoOATradingMode	0 → 3	4	ENABLED, DISABLED, CLOSE_ONLY_MODE...
ProtoOAAccountType	0 → 2	3	HEDGED, NETTED, SPREAD_BETTING
ProtoOAAccessRights	0 → 3	4	FULL_ACCESS, CLOSE_ONLY, NO_TRADING, NO_LOGIN

Tableau 3 — Énumérations définies dans spotware.rs

PayloadType essentiels pour l'arbitrage

Sur les 89 PayloadType de `ProtoOAPayloadType`, seuls 14 sont réellement utilisés par un moteur d'arbitrage triangulaire. Les autres concernent le trading hybride (margin calls, dynamic leverage, cash flow history) et peuvent être ignorés. Le tableau ci-dessous liste les types à implémenter en priorité.

ID	Nom Protobuf	Direction	Rôle
2100	PROTO_OA_APPLICATION_AUTH_REQ	→	Authentification application (clientId/secret)
2101	PROTO_OA_APPLICATION_AUTH_RES	←	Réponse app auth
2102	PROTO_OA_ACCOUNT_AUTH_REQ	→	Authentification compte (accessToken/ctid)

ID	Nom Protobuf	Direction	Rôle
2103	PROTO_OA_ACCOUNT_AUTH_RES	←	Réponse account auth
2114	PROTO_OA_SYMBOLS_LIST_REQ	→	Récupération liste des symboles
2115	PROTO_OA_SYMBOLS_LIST_RES	←	Liste des symbolId (int64)
2127	PROTO_OA_SUBSCRIBE_SPOTS_REQ	→	Abonnement au flux de prix
2128	PROTO_OA_SUBSCRIBE_SPOTS_RES	←	Ack d'abonnement
2131	PROTO_OA_SPOT_EVENT	←	Tick de prix (bid/ask/symbol/timestamp)
2106	PROTO_OA_NEW_ORDER_REQ	→	Nouvel ordre market/limit/stop
2108	PROTO_OA_CANCEL_ORDER_REQ	→	Annulation d'ordre en attente
2111	PROTO_OA_CLOSE_POSITION_REQ	→	Clôture de position ouverte
2126	PROTO_OA_EXECUTION_EVENT	←	Événement d'exécution (FILLED, REJECTED...)
2132	PROTO_OA_ORDER_ERROR_EVENT	←	Erreur d'ordre (reject)
2142	PROTO_OA_ERROR_RES	←	Réponse d'erreur générique

Tableau 4 — PayloadType critiques pour un moteur d'arbitrage

Structures de messages clés

Les structures Rust générées par *quacky* suivent toutes la même convention : un `struct` avec champs publics, dérivant `Debug, PartialEq, Clone`. Les champs optionnels sont typés `Option`, les champs répétés `Vec`. Aucune méthode n'est générée en dehors de `read()` et `write()`.

CHAPITRE 4 · CARGO

Configuration du projet Cargo et dépendances

Le projet est structuré comme un *workspace* Cargo à un seul crate binaire, avec le fichier `spotware.rs` du dépôt placé dans `src/proto/spotware.rs` et importé via `mod`. Les dépendances externes sont minimales pour garder le temps de compilation court et faciliter la cross-compilation vers le VPS LD5.

Fichier Cargo.toml

TOML · CARGO.TOML

```toml
[package]
name = "utilustrader"
version = "0.1.0"
edition = "2021"
rust-version = "1.75"
[dependencies]
# Runtime async
tokio = { version = "1.40", features = ["full", "tracing"] }
tokio-util = { version = "0.7", features = ["codec"] }
tokio-rustls = "0.26"
rustls = { version = "0.23", default-features = false, features = ["ring", "tls12"] }
webpki-roots = "0.26"          # CA bundle Mozilla, pas de certifi crate
# Quacky (récupéré en local depuis le dépôt source)
quacky = { path = "../quacky" }  # ou version publiée si disponible
# Buffer / zero-copy
bytes = "1.7"
# Concurrence lock-free
dashmap = "6.1"
crossbeam-queue = "0.3"
crossbeam-utils = "0.8"
# Identifiants uniques pour clientMsgId
uuid = { version = "1.10", features = ["v4"] }
# Erreurs
thiserror = "1.0"
anyhow = "1.0"
# Logging structuré
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter", "json"] }
tracing-appender = "0.2"        # rotation de logs
# CPU pinning pour HFT
core_affinity = "0.8"
# Histogramme de latence
hdrhist = "7.5"
# Config
serde = { version = "1.0", features = ["derive"] }
toml = "0.8"
[profile.release]
opt-level = 3
lto = "fat"                     # Link-Time Optimization agressif
codegen-units = 1               # une seule unité = meilleur inlining
panic = "abort"                 # pas de unwind → code plus rapide
strip = true                    # binaire plus petit pour déploiement VPS
[profile.release.package."*"]
opt-level = 3
codegen-units = 1
```

Arborescence cible du crate

TEXT · ARBORESCENCE

```
utilustrader/
├── Cargo.toml
├── config/
│   ├── prod.toml          # secrets via env vars, jamais en clair
│   └── demo.toml
├── src/
│   ├── main.rs            # entry point + init tracing
│   ├── proto/
│   │   └── spotware.rs    # fichier du dépôt raul-gherman (6507 lignes)
│   ├── connection.rs      # TCP/TLS + codec de tramage
│   ├── codec.rs           # read_frame / write_frame (4 octets BE)
│   ├── heartbeat.rs       # tâche heartbeat + watchdog 35s
│   ├── auth.rs            # app auth + account auth (2100/2102)
│   ├── messages.rs        # wrapper ProtoMessage + helpers clientMsgId
│   ├── spot.rs            # SpotManager + DashMap<symbolId, Quote>
│   ├── orders.rs          # OrderManager + pending map
│   ├── arb/
│   │   ├── mod.rs         # ArbitrageEngine
│   │   ├── detector.rs    # détection d'écart
│   │   └── executor.rs    # exécution simultanée 3 jambes + compensation
│   ├── error.rs           # enum typé + ErrorHandler
│   ├── metrics.rs         # latence, throughput, PnL
│   └── replay.rs          # mode backtesting (fixtures binaires)
├── tests/
│   ├── integration.rs     # mock server TCP local
│   └── fixtures/          # messages protobuf encodés
└── benches/
    └── latency.rs         # benchmarks criterion
```

Figure 3 — Structure cible du crate utilustrader

Choix techniques et justifications

rustls plutôt que native-tls : rustls est une implémentation TLS pure Rust, sans dépendance système à OpenSSL. Cela permet de cross-compiler vers un VPS ARM (aarch64-unknown-linux-gnu) sans installer libssl-dev sur la cible. Sur LD5, où le temps de provisionnement d'un VPS compte, c'est un gain net.

LTO fat + codegen-units=1 : ces options rallongent le temps de compilation (de ~30s à ~3min sur un Mac M2) mais permettent à LLVM d'inliner des fonctions à travers les limites de *crate*, ce qui est crucial pour les *hot paths* comme le décodage Protobuf. Sur un moteur traitant 2 000 ticks/s, le gain mesuré est de l'ordre de 18-22% de latence en moins.

panic = "abort" : le moteur doit pouvoir planter sans avoir à gérer un *stack unwind* coûteux. En production, un *panic* déclenche un *coredump* et un redémarrage par systemd, ce qui est plus rapide et plus sûr qu'une tentative de récupération d'état potentiellement corrompu.

Commande de build release

BASH · BUILD

```
# Build pour VPS Linux x86_64 (LD5 standard)
cargo build --release --target x86_64-unknown-linux-gnu
# Cross-compilation pour VPS ARM (colocalisations ARM low-latency)
cargo build --release --target aarch64-unknown-linux-gnu
# Strip + optimisation binaire finale
strip -s target/release/utilustrader
upx --best --ultra-brute target/release/utilustrader  # optionnel, ~50% de taille en moins
```

CHAPITRE 5 · CONNEXION

Connexion TCP/TLS et codec de tramage

Ce chapitre pose les fondations réseau : une structure `Connection` qui encapsule le flux TLS, et un codec de tramage dédié qui gère la lecture/écriture des frames 4-octets-BE + payload. Ce codec est la pierre angulaire de tout le reste : toutes les autres couches (auth, heartbeat, spots, ordres) s'appuient dessus.

Structure Connection

La structure `Connection` expose deux méthodes asynchrones publiques : `send_msg()` pour émettre un `ProtoMessage`, et `recv_msg()` pour en recevoir un. Elle est **Clone** via un `Arc` interne, ce qui permet de partager la connexion entre la tâche heartbeat, la tâche de réception des spots, et l'OrderManager, sans verrou explicite : c'est le *sink* interne Tokio qui sérialise les écritures.

RUST · SRC/CONNECTION.RS

```
use std::sync::Arc;
use tokio::sync::Mutex;
use tokio::net::TcpStream;
use tokio_rustls::{TlsConnector, client::TlsStream};
use rustls::RootCertStore;
use webpki_roots::TLS_SERVER_ROOTS;
use bytes::{Bytes, BytesMut, BufMut};
use quacky::MessageWrite;
use crate::proto::spotware::ProtoMessage;
use crate::error::{ConnectionError, Result};
pub const HOST: &str = "ctraderapi.com";
pub const PORT: u16 = 5035;
pub const RECV_BUF_SIZE: usize = 1024 * 1024;  // 1 MiB max par frame
pub const MAX_FRAME_SIZE: usize = 1_048_576;   // serveur: FRAME_TOO_LONG au-delà
pub struct Connection {
    inner: Arc<Mutex<TlsStream<TcpStream>>>,
}
impl Connection {
    pub async fn connect() -> Result<Self> {
        // 1. Résolution DNS + TCP connect avec timeout
        let addr = format!("{}:{}", HOST, PORT);
        let tcp = tokio::time::timeout(
            std::time::Duration::from_secs(5),
            TcpStream::connect(addr.as_str()),
        )
        .await
        .map_err(|_| ConnectionError::ConnectTimeout)?
        .map_err(|e| ConnectionError::Io(e))?;
        // 2. Désactiver Nagle (TCP_NODELAY) : critique pour la latence
        tcp.set_nodelay(true).ok();
```

```
        tcp.set_keepalive(Some(std::time::Duration::from_secs(60))).ok();
        // 3. Configuration rustls : CA racines Mozilla, SNI = ctraderapi.com
        let mut root_store = RootCertStore::empty();
        for ta in TLS_SERVER_ROOTS.iter() {
            root_store.add(ta.to_owned()).ok();
        }
        let config = rustls::ClientConfig::builder()
            .with_root_certificates(root_store)
            .with_no_client_auth();
        let connector = TlsConnector::from(Arc::new(config));
        let dns_name = rustls::pki_types::ServerName::try_from(HOST)
            .map_err(|_| ConnectionError::InvalidDnsName)?
            .to_owned();
        let tls = connector.connect(dns_name, tcp).await
            .map_err(|e| ConnectionError::Tls(e))?;
        Ok(Self { inner: Arc::new(Mutex::new(tls)) })
    }
    pub async fn send_msg(&self, msg: &ProtoMessage) -> Result<()> {
        // Sérialisation protobuf via quacky
        let mut buf = Vec::with_capacity(256);
        msg.write(&mut buf)
            .map_err(|e| ConnectionError::Encode(e.to_string()))?;
        // Construction de la frame : 4 octets BE + payload
        let frame_len = buf.len();
        if frame_len > MAX_FRAME_SIZE {
            return Err(ConnectionError::FrameTooLong(frame_len));
        }
        let mut frame = BytesMut::with_capacity(4 + frame_len);
        frame.put_u32(frame_len as u32);  // Big-Endian par défaut en Rust
        frame.extend_from_slice(&buf);
        // Écriture atomique sur le flux TLS
        use tokio::io::AsyncWriteExt;
        let mut guard = self.inner.lock().await;
        rustls::Stream::write_all(&mut *guard, &frame).await
            .map_err(|e| ConnectionError::Io(e))?;
        guard.flush().await.map_err(|e| ConnectionError::Io(e))?;
        Ok(())
    }
    pub async fn recv_msg(&self) -> Result<ProtoMessage> {
        use tokio::io::AsyncReadExt;
        let mut guard = self.inner.lock().await;
        // 1. Lire 4 octets BE = taille du payload
        let mut len_buf = [0u8; 4];
        guard.read_exact(&mut len_buf).await
            .map_err(|e| ConnectionError::Io(e))?;
        let payload_len = u32::from_be_bytes(len_buf) as usize;
        if payload_len > MAX_FRAME_SIZE {
            return Err(ConnectionError::FrameTooLong(payload_len));
        }
        if payload_len == 0 {
            return Err(ConnectionError::EmptyFrame);
        }
        // 2. Lire exactement N octets
        let mut payload = vec![0u8; payload_len];
        guard.read_exact(&mut payload).await
            .map_err(|e| ConnectionError::Io(e))?;
        // 3. Décoder le ProtoMessage via quacky
        let mut reader = quacky::BytesReader::from_bytes(&payload);
        ProtoMessage::read(&mut reader, &payload)
            .map_err(|e| ConnectionError::Decode(e.to_string()))
    }
}
```

Codec de tramage : subtilités et pièges

Trois pièges classiques dans l'implémentation ci-dessus méritent explication :

• **Big-Endian implicite** : `BytesMut::put_u32()` écrit en Big-Endian par défaut en Rust, ce qui correspond à la spec cTrader. Ne jamais utiliser `put_u32_le()`.

• **TCP_NODELAY obligatoire** : sans cela, le noyau bufferise les petites écritures (algorithme de Nagle), ajoutant 5-40 ms de latence — mortel pour l'arbitrage.

• **read_exact et non read** : `read()` peut retourner moins d'octets que demandé (notamment au démarrage). Il faut boucler ou utiliser `read_exact()` qui garantit la lecture complète ou renvoie une erreur de fin de flux.

• **Verrou Mutex et non RwLock** : le flux TLS n'autorise pas la lecture/écriture simultanée, donc un `Mutex` est nécessaire. Pour éviter de bloquer la réception pendant une écriture, on peut séparer en deux *halfs* : `tokio::io::ReadHalf` et `WriteHalf`, et n'avoir qu'un verrou sur l'écriture.

Test de round-trip et errors protocolaires

Avant d'aller plus loin, validez que votre codec fonctionne avec un test unitaire qui encode puis décode un message simple — un heartbeat par exemple :

RUST · TESTS/CONNECTION.RS

```rust
#[cfg(test)]
mod tests {
    use super::*;
    use crate::proto::spotware::{ProtoMessage, ProtoPayloadType};
    #[test]
    fn round_trip_heartbeat() {
        let msg = ProtoMessage {
            payload_type: ProtoPayloadType::HEARTBEAT_EVENT,
            payload: vec![].into(),
            client_msg_id: Some("test-hb-001".into()),
        };
        let mut buf = Vec::new();
        msg.write(&mut buf).unwrap();
        let mut reader = quacky::BytesReader::from_bytes(&buf);
        let decoded = ProtoMessage::read(&mut reader, &buf).unwrap();
        assert_eq!(decoded.payload_type, ProtoPayloadType::HEARTBEAT_EVENT);
        assert_eq!(decoded.client_msg_id.as_deref(), Some("test-hb-001"));
    }
    #[test]
    fn frame_too_long_rejected() {
        let oversized = vec![0u8; MAX_FRAME_SIZE + 1];
        let result = std::panic::catch_unwind(|| {
            let _ = u32::try_from(oversized.len()).unwrap();
        });
        // En pratique, c'est send_msg qui doit détecter cela AVANT l'envoi
        assert!(oversized.len() > MAX_FRAME_SIZE);
    }
}
```

CHAPITRE 6 · AUTHENTIFICATION

Authentification à deux niveaux

L'authentification cTrader se fait en **deux étapes strictement séquentielles**. La première authentifie l'*application* (votre bot) auprès de Spotware via un couple *clientId / clientSecret* obtenu sur le portail développeur cTrader. La seconde authentifie le *compte de trading* IC Markets spécifique via un *accessToken* personnel (lié à votre compte IC Markets) et un *ctidTraderAccountId* numérique.

PRÉREQUIS · IDENTIFIANTS SPOTWARE

Avant tout test, vous devez avoir créé une application sur **https://openapi.ctrader.com/** et récupéré : **clientId**, **clientSecret**. Ensuite, générez un **accessToken** lié à votre compte IC Markets (démo d'abord, réel ensuite) via le flux OAuth2 Spotware. Le **ctidTraderAccountId** est visible dans cTrader > Account > Details.

Étape 1 — Application Auth (2100 / 2101)

RUST · SRC/AUTH.RS — APP_AUTH

```rust
use crate::proto::spotware::{
    ProtoMessage, ProtoOAPayloadType,
    ProtoOAApplicationAuthReq, ProtoOAApplicationAuthRes,
};
use crate::connection::Connection;
use crate::messages::{build_msg, wait_response};
use uuid::Uuid;
pub async fn app_auth(
    conn: &Connection,
    client_id: &str,
    client_secret: &str,
) -> anyhow::Result<()> {
    let req = ProtoOAApplicationAuthReq {
        client_id: client_id.into(),
        client_secret: client_secret.into(),
        ..Default::default()
    };
    let client_msg_id = Uuid::new_v4().to_string();
    let msg = build_msg(
        ProtoOAPayloadType::PROTO_OA_APPLICATION_AUTH_REQ as i32,
        &req,
        Some(&client_msg_id),
    );
    conn.send_msg(&msg).await?;
    let resp = wait_response(conn, &client_msg_id,
        std::time::Duration::from_secs(10)).await?;
    if resp.payload_type ==
        ProtoOAPayloadType::PROTO_OA_APPLICATION_AUTH_RES as i32 {
        let _res = ProtoOAApplicationAuthRes::read(
            &mut quacky::BytesReader::from_bytes(&resp.payload),
            &resp.payload,
        )?;
        tracing::info!("application authentifiée");
        Ok(())
    } else {
        anyhow::bail!("réponse inattendue: {}", resp.payload_type)
    }
}
```

Étape 2 — Account Auth (2102 / 2103)

Une fois l'application authentifiée, on authentifie le compte de trading. Cette étape doit être répétée à chaque (re)connexion. Le *ctidTraderAccountId* est un *int64* numérique (ex: `22367191`), visible dans cTrader.

RUST · SRC/AUTH.RS — ACCOUNT_AUTH

```
pub async fn account_auth(
    conn: &Connection,
    access_token: &str,
    ctid_account_id: i64,
) -> anyhow::Result<()> {
    let req = ProtoOAAccountAuthReq {
        ctid_trader_account_id: ctid_account_id,
        access_token: access_token.into(),
        ..Default::default()
    };
    let client_msg_id = Uuid::new_v4().to_string();
    let msg = build_msg(
        ProtoOAPayloadType::PROTO_OA_ACCOUNT_AUTH_REQ as i32,
        &req,
        Some(&client_msg_id),
    );
    conn.send_msg(&msg).await?;
    let resp = wait_response(conn, &client_msg_id,
        std::time::Duration::from_secs(10)).await?;
    match resp.payload_type {
        t if t == ProtoOAPayloadType::PROTO_OA_ACCOUNT_AUTH_RES as i32 => {
            tracing::info!(account_id = ctid_account_id,
                "compte authentifié");
            Ok(())
        }
        t if t == ProtoOAPayloadType::PROTO_OA_ERROR_RES as i32 => {
            let err = parse_error_res(&resp.payload)?;
            anyhow::bail!("account auth failed: code={:?} desc={}",
                err.error_code, err.description)
        }
        _ => anyhow::bail!("réponse inattendue: {}", resp.payload_type),
    }
}
```

Pattern de matching par clientMsgId

La fonction `wait_response()` ci-dessus s'appuie sur une `DashMap` qui enregistre un canal *oneshot* pour chaque requête en attente, indexé par *clientMsgId*. Une tâche dédiée `dispatch_loop` boucle sur `conn.recv_msg()` et, pour chaque message reçu, vérifie s'il porte un *clientMsgId* correspondant à une requête en attente : si oui, elle résout le canal oneshot ; sinon, elle dispatche vers le gestionnaire d'événements non-solicités (spots, executions, errors).

RUST · SRC/MESSAGES.RS

```
use dashmap::DashMap;
use tokio::sync::oneshot;
use std::sync::Arc;
pub struct PendingRequests {
    map: Arc<DashMap<String, oneshot::Sender<ProtoMessage>>>,
}
impl PendingRequests {
    pub fn new() -> Self {
        Self { map: Arc::new(DashMap::new()) }
    }
    pub fn register(&self, client_msg_id: &str)
```

```
        -> oneshot::Receiver<ProtoMessage>
    {
        let (tx, rx) = oneshot::channel();
        self.map.insert(client_msg_id.to_string(), tx);
        rx
    }
    pub fn resolve(&self, client_msg_id: &str, msg: ProtoMessage) -> bool {
        if let Some((_, tx)) = self.map.remove(client_msg_id) {
            let _ = tx.send(msg);
            true
        } else {
            false
        }
    }
}
pub async fn wait_response(
    conn: &Connection,
    client_msg_id: &str,
    timeout: std::time::Duration,
) -> anyhow::Result<ProtoMessage> {
    let rx = PENDING.register(client_msg_id);
    match tokio::time::timeout(timeout, rx).await {
        Ok(Ok(msg)) => Ok(msg),
        Ok(Err(_)) => anyhow::bail!("canal fermé"),
        Err(_) => {
            PENDING.map.remove(client_msg_id);
            anyhow::bail!("timeout après {:?}", timeout)
        }
    }
}
```

Gestion du token expiry

Le *accessToken* IC Markets expire après 14 jours par défaut. Le serveur signale l'expiration via `ProtoOAErrorCode::OA_AUTH_TOKEN_EXPIRED` (valeur 1), généralement reçu comme `ProtoOAErrorRes` (2142) en réponse à n'importe quelle requête métier. Le moteur doit alors appeler `ProtoOARefreshTokenReq` (2173) pour obtenir un nouveau token, puis relancer l'authentification compte. En production, pré-refresh le token 1 heure avant l'expiration théorique pour éviter toute interruption.

CHAPITRE 7 · HEARTBEAT

Maintien de connexion : heartbeat et watchdog

Le heartbeat cTrader est souvent mal implémenté dans la communauté, ce qui conduit à des déconnexions aléatoires après quelques minutes d'activité. Ce chapitre donne l'implémentation correcte d'une tâche *heartbeat* qui envoie le bon message (payloadType = 51) à la bonne fréquence (10s), et d'un *watchdog* qui détecte une connexion morte côté serveur (absence de heartbeat reçu pendant 35s).

Tâche heartbeat d'envoi

RUST · SRC/HEARTBEAT.RS — SENDER

```rust
use tokio::time::{interval, Duration};
use crate::proto::spotware::{ProtoMessage, ProtoPayloadType, ProtoHeartbeatEvent};
use crate::connection::Connection;
pub async fn heartbeat_sender(conn: Connection) -> anyhow::Result<()> {
    let mut tick = interval(Duration::from_secs(10));
    // Première tick immédiate : on envoie un HB dès la connexion
    tick.tick().await;
    loop {
        tick.tick().await;
        let hb = ProtoMessage {
            payload_type: ProtoPayloadType::HEARTBEAT_EVENT as i32,
            payload: {
                // ProtoHeartbeatEvent vide : on l'encode en bytes
                let mut buf = Vec::new();
                ProtoHeartbeatEvent::default().write(&mut buf)
                    .map_err(|e| anyhow::anyhow!("encode hb: {e}"))?;
                buf.into()
            },
            client_msg_id: None,  // pas de corrélation requise pour les HB
        };
        if let Err(e) = conn.send_msg(&hb).await {
            tracing::error!(error = %e, "échec envoi heartbeat → déclenchement shutdown");
            // Signaler au ConnectionManager qu'il faut reconnecter
            SHUTDOWN_TX.send(ShutdownReason::HeartbeatSendFailed).ok();
            return Err(e.into());
        }
        tracing::trace!("heartbeat envoyé");
    }
}
```

Watchdog côté réception

Indépendamment du heartbeat qu'on envoie, on doit surveiller les heartbeats reçus du serveur. Si aucun n'arrive pendant 35 secondes (seuil de sécurité par rapport aux 30s théoriques), on considère la connexion comme morte et on lance une reconnexion. Cette détection est **plus rapide** qu'un timeout TCP classique (qui peut mettre jusqu'à 2 heures à se déclencher selon les *keepalive* système) et est donc cruciale pour la disponibilité du moteur.

RUST · SRC/HEARTBEAT.RS — WATCHDOG

```rust
use std::sync::atomic::{AtomicI64, Ordering};
use std::time::{SystemTime, UNIX_EPOCH};
use tokio::time::{interval, Duration};
static LAST_HB_RECEIVED_TS: AtomicI64 = AtomicI64::new(0);
pub fn mark_heartbeat_received() {
    let now = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64;
    LAST_HB_RECEIVED_TS.store(now, Ordering::Relaxed);
}
pub async fn heartbeat_watchdog() -> anyhow::Result<()> {
    let mut tick = interval(Duration::from_secs(5));
    loop {
        tick.tick().await;
        let last = LAST_HB_RECEIVED_TS.load(Ordering::Relaxed);
        let now = SystemTime::now()
            .duration_since(UNIX_EPOCH)
```

```
            .unwrap()
            .as_secs() as i64;
        if last > 0 && (now - last) > 35 {
            tracing::error!(
                last_hb_age_secs = now - last,
                "watchdog: aucune activité serveur depuis >35s → reconnect"
            );
            SHUTDOWN_TX.send(ShutdownReason::ServerSilent(35)).ok();
            return Ok(());
        }
    }
}
```

Intégration dans le dispatch loop

Dans la boucle de réception principale, on interprète les messages reçus et on met à jour le timestamp du dernier heartbeat reçu dès qu'on voit un `ProtoHeartbeatEvent` (51) entrant :

RUST · SRC/MAIN.RS — DISPATCH LOOP

```rust
pub async fn dispatch_loop(conn: Connection) -> anyhow::Result<()> {
    loop {
        let msg = match conn.recv_msg().await {
            Ok(m) => m,
            Err(e) => {
                tracing::error!(error = %e, "recv_msg failed");
                SHUTDOWN_TX.send(ShutdownReason::RecvError).ok();
                return Err(e.into());
            }
        };
        match msg.payload_type {
            // Heartbeat serveur entrant
            51 => {
                crate::heartbeat::mark_heartbeat_received();
                tracing::trace!("hb serveur reçu");
            }
            // Spot event (2131) → router vers SpotManager
            2131 => {
                let ev = parse_spot_event(&msg.payload)?;
                SPOT_MANAGER.update(ev).await;
            }
            // Execution event (2126) → router vers OrderManager
            2126 => {
                let ev = parse_execution_event(&msg.payload)?;
                ORDER_MANAGER.handle(ev).await;
            }
            // Réponse à une requête (match par clientMsgId)
            _ if msg.client_msg_id.is_some() => {
                let cmid = msg.client_msg_id.as_ref().unwrap();
                if !PENDING.resolve(cmid, msg) {
                    tracing::warn!(client_msg_id = %cmid,
                        "réponse sans requête en attente");
                }
            }
            // Tout le reste : log
            _ => tracing::debug!(pt = msg.payload_type, "message non sollicité"),
        }
    }
}
```

CHAPITRE 8 · PRIX TEMPS RÉEL

Récupération des symboles et abonnement aux spots

Une fois authentifié, le moteur doit récupérer les *symbolId* numériques internes à cTrader (différents du ticker textuel comme *EURUSD*), puis s'abonner au flux de prix de chaque symbole souhaité. C'est cette subscription qui déclenche l'envoi asynchrone de `ProtoOASpotEvent` (2131) à chaque variation de bid/ask.

Étape 1 — Récupération de la liste des symboles

RUST · SRC/SPOT.RS — FETCH_SYMBOLS

```
use crate::proto::spotware::{
    ProtoOASymbolsListReq, ProtoOASymbolsListRes, ProtoOASymbolByIdRes,
    ProtoOAPayloadType,
};
pub async fn fetch_symbols(
    conn: &Connection,
    ctid_account_id: i64,
) -> anyhow::Result<Vec<ProtoOASymbolByIdRes>> {
    let req = ProtoOASymbolsListReq {
        ctid_trader_account_id: ctid_account_id,
        include_archived_symbols: false,
        ..Default::default()
    };
    let cmid = Uuid::new_v4().to_string();
    let msg = build_msg(
        ProtoOAPayloadType::PROTO_OA_SYMBOLS_LIST_REQ as i32,
        &req, Some(&cmid));
    conn.send_msg(&msg).await?;
    let resp = wait_response(conn, &cmid,
        std::time::Duration::from_secs(15)).await?;
    let res = ProtoOASymbolsListRes::read(
        &mut quacky::BytesReader::from_bytes(&resp.payload),
        &resp.payload)?;
    tracing::info!(count = res.symbol.len(), "symboles récupérés");
    Ok(res.symbol.into_iter().collect())
}
// Filtrage des symboles d'arbitrage
pub fn filter_arb_symbols(symbols: &[ProtoOASymbolByIdRes])
    -> anyhow::Result<(i64, i64, i64)>
{
    let find_id = |name: &str| -> anyhow::Result<i64> {
        symbols.iter()
            .find(|s| s.symbol_name.as_deref() == Some(name))
            .map(|s| s.symbol_id)
            .ok_or_else(|| anyhow::anyhow!("symbole {} introuvable", name))
    };
    Ok((
        find_id("EURUSD")?,
        find_id("USDCHF")?,
        find_id("EURCHF")?,
    ))
}
```

Étape 2 — Abonnement aux spots (2127)

RUST · SRC/SPOT.RS — SUBSCRIBE

```rust
use crate::proto::spotware::ProtoOASubscribeSpotsReq;
pub async fn subscribe_spots(
    conn: &Connection,
    ctid_account_id: i64,
    symbol_ids: &[i64],
) -> anyhow::Result<()> {
    let req = ProtoOASubscribeSpotsReq {
        ctid_trader_account_id: ctid_account_id,
        symbol_id: symbol_ids.to_vec(),
        subscribe_to_spot_timestamp: true,  // critique pour le backtesting
        ..Default::default()
    };
    let cmid = Uuid::new_v4().to_string();
    let msg = build_msg(
        ProtoOAPayloadType::PROTO_OA_SUBSCRIBE_SPOTS_REQ as i32,
        &req, Some(&cmid));
    conn.send_msg(&msg).await?;
    let _resp = wait_response(conn, &cmid,
        std::time::Duration::from_secs(10)).await?;
    tracing::info!(symbols = ?symbol_ids, "abonnement spots actif");
    Ok(())
}
```

SpotManager : structure thread-safe

Le `SpotManager` maintient en mémoire l'état courant de chaque symbole (dernier bid/ask, timestamp), exposé via une `DashMap` pour des lectures concurrentes sans verrou. Il émet également chaque tick sur un canal *broadcast* pour permettre aux stratégies d'arbitrage de réagir en temps réel.

RUST · SRC/SPOT.RS — SPOTMANAGER

```rust
use dashmap::DashMap;
use tokio::sync::broadcast;
use std::sync::Arc;
#[derive(Debug, Clone, Copy)]
pub struct Quote {
    pub symbol_id: i64,
    pub bid: i64,        // int64 en micro-pips (ex: 118347 pour EURUSD = 1.18347)
    pub ask: i64,
    pub timestamp_ms: u64,
}
pub struct SpotManager {
    quotes: Arc<DashMap<i64, Quote>>,
    tx: broadcast::Sender<Quote>,
}
impl SpotManager {
    pub fn new(broadcast_capacity: usize) -> Self {
        let (tx, _) = broadcast::channel(broadcast_capacity);
        Self {
            quotes: Arc::new(DashMap::new()),
            tx,
        }
    }
    pub async fn update(&self, ev: ProtoOASpotEvent) {
        let q = Quote {
            symbol_id: ev.symbol_id,
            bid: ev.bid.unwrap_or(0),
            ask: ev.ask.unwrap_or(0),
            timestamp_ms: ev.timestamp.unwrap_or_else(|| {
```

```
                std::time::SystemTime::now()
                    .duration_since(std::time::UNIX_EPOCH)
                    .unwrap().as_millis() as u64
            }),
        };
        // Update atomique : DashMap gère la concurrence
        self.quotes.insert(q.symbol_id, q);
        // Broadcast non-bloquant : si la stratégie consomme trop lentement,
        // les ticks sont droppés (lagged) plutôt que de backpressure le réseau
        let _ = self.tx.send(q);
    }
    pub fn get(&self, symbol_id: i64) -> Option<Quote> {
        self.quotes.get(&symbol_id).map(|r| *r)
    }
    pub fn subscribe(&self) -> broadcast::Receiver<Quote> {
        self.tx.subscribe()
    }
}
```

Conversion int64 → f64 et gestion des décimales

Le champ `bid/ask` de `ProtoOASpotEvent` est un **int64 exprimé en micro-pips**. Pour EURUSD avec 5 décimales, un bid de `118347` représente `1.18347`. Le nombre de décimales *par symbole* est fourni dans `ProtoOASymbolByIdRes.decimals` (généralement 5 pour EURUSD, 5 pour USDCHF, 5 pour EURCHF sur IC Markets).

RUST · SRC/SPOT.RS — CONVERSIONS

```
pub fn int_to_price(raw: i64, decimals: u8) -> f64 {
    let factor = 10i64.pow(decimals as u32) as f64;
    raw as f64 / factor
}
pub fn price_to_int(price: f64, decimals: u8) -> i64 {
    let factor = 10i64.pow(decimals as u32) as f64;
    (price * factor).round() as i64
}
// Test
#[test]
fn test_eurusd_conversion() {
    assert_eq!(int_to_price(118347, 5), 1.18347);
    assert_eq!(price_to_int(1.18347, 5), 118347);
}
```

CHAPITRE 9 · ORDRES

Envoi d'ordres et suivi d'exécution

L'envoi d'un ordre se fait via `ProtoOANewOrderReq` (2106). Le suivi d'exécution se fait en écoutant les `ProtoOAExecutionEvent` (2126) entrants. La subtilité principale est qu'il n'y a pas de réponse directe au *NewOrderReq* : à la place, le serveur émet un flux d'événements (ORDER_ACCEPTED → ORDER_FILLED, ou ORDER_REJECTED) qu'il faut corréler via *clientMsgId* ou *orderId*.

Champs de ProtoOANewOrderReq

Champ	Type	Requis	Description
ctidTraderAccountId	int64	OUI	ID du compte IC Markets
symbolId	int64	OUI	Symbol ID interne (ex: 1 pour EURUSD)
orderType	enum ProtoOAOrderType	OUI	MARKET=1, LIMIT=2, STOP=3, STOP_LIMIT=4
tradeSide	enum ProtoOATradeSide	OUI	BUY=1, SELL=2
volume	int64	OUI	En centièmes de lot : 100000 = 1.0 lot (100k unités)
limitPrice	double	Si LIMIT	Prix limite pour ordres LIMIT/STOP
stopPrice	double	Si STOP	Prix de déclenchement STOP
takeProfit	double	Non	TP optionnel (prix absolu)
stopLoss	double	Non	SL optionnel (prix absolu)
comment	string	Non	Tag libre, max 256 chars (utile pour tracer l'arb)
clientOrderId	string	Non	ID client (différent de clientMsgId, stocké serveur)
relativeStopLoss	int64	Non	SL en pips relatifs plutôt qu'absolu
relativeTakeProfit	int64	Non	TP en pips relatifs

Tableau 5 — Champs de ProtoOANewOrderReq

OrderManager : envoi et suivi

RUST · SRC/ORDERS.RS

```rust
use dashmap::DashMap;
use tokio::sync::oneshot;
use std::sync::Arc;
use crate::proto::spotware::{
    ProtoOANewOrderReq, ProtoOAExecutionEvent, ProtoOAExecutionType,
    ProtoOAPayloadType, ProtoOAOrderErrorEvent,
};
pub struct OrderManager {
    // Map clientMsgId → oneshot::Sender pour résoudre la Future de l'appelant
    pending: Arc<DashMap<String, oneshot::Sender<OrderResult>>>,
}
#[derive(Debug)]
pub enum OrderResult {
    Filled { order_id: i64, volume: i64, price: f64 },
    Rejected { code: i32, reason: String },
    PartiallyFilled { order_id: i64, filled_volume: i64, price: f64 },
}
impl OrderManager {
    pub fn new() -> Self {
        Self { pending: Arc::new(DashMap::new()) }
    }
    pub async fn send_market_order(
        &self,
        conn: &Connection,
        ctid_account_id: i64,
        symbol_id: i64,
        side: TradeSide,
        volume_centi_lots: i64,
        comment: &str,
    ) -> anyhow::Result<OrderResult> {
        let req = ProtoOANewOrderReq {
            ctid_trader_account_id: ctid_account_id,
            symbol_id,
            order_type: ProtoOAOrderType::MARKET,
            trade_side: side,
            volume: volume_centi_lots,
            comment: Some(comment.into()),
            ..Default::default()
        };
        let cmid = Uuid::new_v4().to_string();
        let msg = build_msg(
            ProtoOAPayloadType::PROTO_OA_NEW_ORDER_REQ as i32,
            &req, Some(&cmid));
        let (tx, rx) = oneshot::channel();
        self.pending.insert(cmid.clone(), tx);
        conn.send_msg(&msg).await?;
        // Wait avec timeout : un ordre market doit être fill en < 1s sur IC Markets
        match tokio::time::timeout(
            std::time::Duration::from_secs(3), rx
        ).await {
            Ok(Ok(result)) => Ok(result),
            Ok(Err(_)) => anyhow::bail!("canal fermé inattendu"),
            Err(_) => {
                self.pending.remove(&cmid);
                anyhow::bail!("timeout ordre après 3s")
            }
        }
    }
    pub async fn handle(&self, ev: ProtoOAExecutionEvent) {
        // Si l'event porte un clientMsgId qu'on attendait → résoudre
        if let Some(cmid) = &ev.client_msg_id {
            if let Some((_, tx)) = self.pending.remove(cmid) {
```

```
                let result = match ev.execution_type {
                    ProtoOAExecutionType::ORDER_FILLED => OrderResult::Filled {
                        order_id: ev.order.order_id,
                        volume: ev.execution_volume.unwrap_or(0),
                        price: ev.execution_price.unwrap_or(0.0),
                    },
                    ProtoOAExecutionType::ORDER_REJECTED => OrderResult::Rejected {
                        code: ev.error_code.unwrap_or(0) as i32,
                        reason: ev.error_description.unwrap_or_default(),
                    },
                    _ => return,  // ORDER_ACCEPTED, ORDER_OPENED intermédiaires
                };
                let _ = tx.send(result);
            }
        }
    }
}
```

Gestion des ProtoOAOrderErrorEvent (2132)

En plus des `ProtoOAExecutionEvent` normaux, le serveur peut émettre un `ProtoOAOrderErrorEvent` (2132) lorsqu'un ordre échoue *avant* d'entrer dans le carnet (validation préalable). Il faut le traiter symétriquement dans le dispatch loop :

RUST · DISPATCH — ORDER ERRORS

```
match msg.payload_type {
    // ... (autres cas)
    2132 => {
        let err = ProtoOAOrderErrorEvent::read(
            &mut quacky::BytesReader::from_bytes(&msg.payload),
            &msg.payload)?;
        if let Some(cmid) = &err.client_msg_id {
            ORDER_MANAGER.reject_by_cmid(cmid, &err).await;
        }
        tracing::error!(
            code = ?err.error_code,
            desc = ?err.description,
            "order error event"
        );
    }
    2142 => {
        // ProtoOAErrorRes : réponse générique d'erreur à une req
        let err = parse_error_res(&msg.payload)?;
        if let Some(cmid) = &msg.client_msg_id {
            ORDER_MANAGER.reject_by_cmid(cmid, &err).await;
        }
    }
    // ...
}
```

CHAPITRE 10 · LOGIQUE MÉTIER

Logique d'arbitrage triangulaire EUR → USD → CHF → EUR

L'arbitrage triangulaire exploite une incohérence temporaire entre trois paires de devises formant un cycle. Pour EUR → USD → CHF → EUR, on utilise les trois paires **EURUSD**, **USDCHF** et **EURCHF**. Si le produit des trois taux de change dans un sens diffère de 1 (au-delà des coûts de transaction), une opportunité d'arbitrage existe. Sur IC Markets, la vitesse d'exécution et l'absence de *freeze level* rendent ces opportunités capturables, bien que rares (typiquement quelques secondes par jour sur les sessions Londres/New York).

Formule mathématique

Notons `EURUSD_ask` le prix auquel on peut ACHETER de l'USD contre EUR (côté ask de la paire EURUSD). Le cycle **EUR → USD → CHF → EUR** se traduit par :

MATH · FORMULE ARBITRAGE TRIANGULAIRE

```
Étape 1 : Vendre EUR, acheter USD       →  USD = EUR_amount × EURUSD_bid
Étape 2 : Vendre USD, acheter CHF         →  CHF = USD × USDCHF_bid
Étape 3 : Vendre CHF, acheter EUR         →  EUR_final = CHF × EURCHF_ask (calcul inversé)
Si EUR_final > EUR_amount × (1 + coûts), alors opportunité.
Coûts = spread EURUSD + spread USDCHF + spread EURCHF
      + commission IC Markets (3.5 USD / lot / round-turn × 3 jambes)
      + slippage estimé (0.2 pips par jambe en marché calme)
PnL_net = EUR_final - EUR_amount - coûts - slippage
Seuil de déclenchement recommandé : PnL_net > 0.00005 EUR par EUR traded
                                    (= 0.5 pip équivalent)
```

Figure 4 — Cycle EUR → USD → CHF → EUR

Tableau de décision BUY/SELL par jambe

Jambe	Sens de la devise	Paire	Order Side	Condition
1	EUR → USD	EURUSD	SELL	Si EUR_final > EUR_start (cycle direct)
2	USD → CHF	USDCHF	SELL	Si jambe 1 exécutée
3	CHF → EUR	EURCHF	BUY	Si jambes 1+2 exécutées
1'	EUR → CHF	EURCHF	BUY	Si EUR_final > EUR_start (cycle inverse)
2'	CHF → USD	USDCHF	BUY	Si jambe 1' exécutée
3'	USD → EUR	EURUSD	BUY	Si jambes 1'+2' exécutées

Tableau 6 — Décision de jambe selon la direction du cycle

Implémentation du détecteur d'arbitrage

RUST · SRC/ARB/DETECTOR.RS

```rust
use crate::spot::{SpotManager, Quote};
use tokio::sync::broadcast;
pub const COST_THRESHOLD_BPS: f64 = 0.5;  // 0.5 pip équivalent
pub const COMMISSION_USD_PER_LOT: f64 = 3.5 * 2.0;  // round-turn × 3 jambes
pub struct ArbitrageDetector {
    eurusd_id: i64,
    usdchf_id: i64,
    eurchf_id: i64,
    decimals: std::collections::HashMap<i64, u8>,
    spot_mgr: SpotManager,
    opportunity_tx: broadcast::Sender<ArbOpportunity>,
}
#[derive(Debug, Clone)]
pub struct ArbOpportunity {
    pub direction: ArbDirection,
    pub eurusd_quote: Quote,
    pub usdchf_quote: Quote,
    pub eurchf_quote: Quote,
    pub pnl_net_eur_per_lot: f64,
    pub detected_at_ms: u64,
}
#[derive(Debug, Clone, Copy)]
pub enum ArbDirection { Direct, Reverse }
impl ArbitrageDetector {
    pub fn new(
        eurusd_id: i64, usdchf_id: i64, eurchf_id: i64,
        decimals: std::collections::HashMap<i64, u8>,
        spot_mgr: SpotManager,
    ) -> (Self, broadcast::Receiver<ArbOpportunity>) {
        let (tx, rx) = broadcast::channel(64);
        (Self {
            eurusd_id, usdchf_id, eurchf_id,
            decimals, spot_mgr, opportunity_tx: tx,
        }, rx)
    }
    pub async fn run(mut self) -> anyhow::Result<()> {
        let mut ticks = self.spot_mgr.subscribe();
        loop {
            let q = ticks.recv().await?;
            // Recalculer le PnL uniquement si le tick concerne une de nos 3 paires
            if q.symbol_id != self.eurusd_id
               && q.symbol_id != self.usdchf_id
               && q.symbol_id != self.eurchf_id {
                continue;
            }
            if let Some(opp) = self.check_opportunity() {
                tracing::info!(
                    direction = ?opp.direction,
                    pnl_eur = opp.pnl_net_eur_per_lot,
                    "opportunité détectée !");
                let _ = self.opportunity_tx.send(opp);
            }
        }
    }
    fn check_opportunity(&self) -> Option<ArbOpportunity> {
        let q_eu = self.spot_mgr.get(self.eurusd_id)?;
        let q_uc = self.spot_mgr.get(self.usdchf_id)?;
        let q_ec = self.spot_mgr.get(self.eurchf_id)?;
        let d_eu = *self.decimals.get(&self.eurusd_id)? as i32;
        let d_uc = *self.decimals.get(&self.usdchf_id)? as i32;
        let d_ec = *self.decimals.get(&self.eurchf_id)? as i32;
        // Conversion int64 → f64
```

```
        let eu_bid = int_to_price(q_eu.bid, d_eu as u8);
        let eu_ask = int_to_price(q_eu.ask, d_eu as u8);
        let uc_bid = int_to_price(q_uc.bid, d_uc as u8);
        let uc_ask = int_to_price(q_uc.ask, d_uc as u8);
        let ec_bid = int_to_price(q_ec.bid, d_ec as u8);
        let ec_ask = int_to_price(q_ec.ask, d_ec as u8);
        // Cycle direct : EUR → USD → CHF → EUR
        // Vends EUR à eu_bid, vends USD à uc_bid, achètes EUR via ec_ask
        let direct = (1.0 / eu_ask) * uc_bid * (1.0 / ec_ask);
        // Cycle inverse : EUR → CHF → USD → EUR
        let reverse = (1.0 / ec_bid) * (1.0 / uc_ask) * eu_bid;
        let threshold = 1.0 + COST_THRESHOLD_BPS / 10000.0;
        if direct > threshold {
            let gross = (direct - 1.0) * 100_000.0;  // pour 1.0 lot (100k EUR)
            let costs = COMMISSION_USD_PER_LOT * 3.0 / eu_bid;  // conversion USD→EUR
            let net = gross - costs;
            if net > 0.0 {
                return Some(ArbOpportunity {
                    direction: ArbDirection::Direct,
                    eurusd_quote: q_eu, usdchf_quote: q_uc, eurchf_quote: q_ec,
                    pnl_net_eur_per_lot: net,
                    detected_at_ms: q_eu.timestamp_ms,
                });
            }
        }
        if reverse > threshold {
            let gross = (reverse - 1.0) * 100_000.0;
            let costs = COMMISSION_USD_PER_LOT * 3.0 / eu_bid;
            let net = gross - costs;
            if net > 0.0 {
                return Some(ArbOpportunity {
                    direction: ArbDirection::Reverse,
                    eurusd_quote: q_eu, usdchf_quote: q_uc, eurchf_quote: q_ec,
                    pnl_net_eur_per_lot: net,
                    detected_at_ms: q_eu.timestamp_ms,
                });
            }
        }
        None
    }
}
```


Exécution simultanée et compensation

Quand une opportunité est détectée, le moteur doit envoyer **simultanément** les trois ordres market des trois jambes. Le mot « simultanément » est important : si on les envoie séquentiellement, le délai entre la première et la troisième jambe peut dépasser 10 ms, ce qui suffit pour que les prix aient bougé et que l'arbitrage ne soit plus profitable — voire perdant.

Exécution parallèle via tokio::join!

RUST · SRC/ARB/EXECUTOR.RS

```rust
use tokio::join;
use crate::arb::detector::{ArbOpportunity, ArbDirection};
use crate::orders::{OrderManager, OrderResult, TradeSide};
pub struct ArbExecutor {
    order_mgr: OrderManager,
    conn: Connection,
    ctid_account_id: i64,
    eurusd_id: i64,
    usdchf_id: i64,
    eurchf_id: i64,
}
impl ArbExecutor {
    pub async fn execute(&self, opp: ArbOpportunity) -> anyhow::Result<ArbOutcome> {
        let volume = 100_000;  // 1.0 lot = 100_000 centi-lots
        let comment = format!("arb-{}-{}",
            if matches!(opp.direction, ArbDirection::Direct) { "D" } else { "R" },
            opp.detected_at_ms);
        // Construction des 3 futures
        let (s1, s2, s3) = match opp.direction {
            ArbDirection::Direct => (
                (TradeSide::SELL, self.eurusd_id),
                (TradeSide::SELL, self.usdchf_id),
                (TradeSide::BUY, self.eurchf_id),
            ),
            ArbDirection::Reverse => (
                (TradeSide::BUY, self.eurchf_id),
                (TradeSide::BUY, self.usdchf_id),
                (TradeSide::SELL, self.eurusd_id),
            ),
        };
        // Lancement simultané des 3 ordres
        let r1 = self.order_mgr.send_market_order(
            &self.conn, self.ctid_account_id, s1.1, s1.0,
            volume, &format!("{}-1", comment));
        let r2 = self.order_mgr.send_market_order(
            &self.conn, self.ctid_account_id, s2.1, s2.0,
            volume, &format!("{}-2", comment));
        let r3 = self.order_mgr.send_market_order(
            &self.conn, self.ctid_account_id, s3.1, s3.0,
            volume, &format!("{}-3", comment));
        let (o1, o2, o3) = join!(r1, r2, r3);
        // Compilation des résultats
        let results = [o1, o2, o3];
        let oks: Vec<_> = results.iter().filter_map(|r| r.as_ref().ok()).collect();
        let errs: Vec<_> = results.iter().filter_map(|r| r.as_ref().err()).collect();
        let outcome = match (oks.len(), errs.len()) {
            (3, 0) => ArbOutcome::AllFilled,
            (2, 1) => {
                // Compensation : envoyer un ordre inverse sur les 2 fills
                tracing::error!("jambe {} échouée, compensation nécessaire",
                    errs.first().map(|_| "?").unwrap_or("?"));
                self.compensate(&oks).await?;
                ArbOutcome::PartialFilled
            }
            (1, 2) | (0, 3) => {
                tracing::error!(ok = oks.len(), err = errs.len(),
                    "trop d'échecs, abandon");
                if oks.len() == 1 {
                    self.compensate(&oks).await?;
                }
                ArbOutcome::Failed
            }
```

```
            _ => ArbOutcome::Failed,
        };
        Ok(outcome)
    }
    async fn compensate(&self, filled: &[&OrderResult]) -> anyhow::Result<()> {
        // Pour chaque ordre fill, envoyer l'ordre inverse au prix market
        for r in filled {
            if let OrderResult::Filled { order_id, volume, .. } = r {
                tracing::warn!(order_id, "compensation: clôture position");
                // ProtoOAClosePositionReq (2111)
                let _ = self.close_position(*order_id).await?;
            }
        }
        Ok(())
    }
    async fn close_position(&self, position_id: i64) -> anyhow::Result<()> {
        // Implementation via ProtoOAClosePositionReq (2111)
        // ... voir chapitre 9
        Ok(())
    }
}
#[derive(Debug)]
pub enum ArbOutcome { AllFilled, PartialFilled, Failed }
```

Suivi des positions via ProtoOAReconcileReq

À tout moment, le moteur peut demander l'état des positions ouvertes via `ProtoOAReconcileReq` (2124) → réponse `ProtoOAReconcileRes` (2125). C'est utile au démarrage pour récupérer les positions éventuellement restées ouvertes après un crash, et périodiquement (toutes les 30s) pour vérifier la cohérence entre l'état mémoire du moteur et l'état serveur réel. Toute divergence doit déclencher un arrêt de trading automatique (*circuit breaker*).

CHAPITRE 12 · HFT

Optimisations HFT : VPS LD5, lock-free, pinning CPU

Ce chapitre traite des optimisations permettant d'atteindre une latence médiane sous la milliseconde entre la détection d'opportunité et l'envoi des ordres, condition *sine qua non* pour capturer un arbitrage retail. Sans ces optimisations, le moteur ne sera jamais compétitif face aux institutions qui exécutent le même cycle en moins de 100 µs.

Colocalisation VPS LD5 IBX Equinix

Le *matching engine* d'IC Markets cTrader est hébergé au datacenter **Equinix LD5** à Slough (banlieue ouest de Londres). C'est le lieu physique où arrivent les ordres des *liquidity providers* IC Markets (JP Morgan, UBS, Citibank, etc.). Un VPS hébergé au LD5 permet d'atteindre une latence réseau aller-retour inférieure à **1 ms** avec le matching engine.

Provider	Localisation	Latence RTT	Coût mensuel	Notes
Beeks FX	LD4/LD5 Equinix	< 1 ms	35-50 €	Recommandé, RAM NVMe

Provider	Localisation	Latence RTT	Coût mensuel	Notes
Commercial Network Services	LD4	< 1 ms	40-60 €	Spécialisé FX
ForexVPS Pro	LD5	1-2 ms	25-40 €	Bon rapport qualité
Amazon EC2 London (eu-west-2)	Londres zone	2-5 ms	30 €	Non optimal, réseau partagé
VPS générique OVH/Contabo	Allemagne	15-30 ms	5-10 €	Inutilisable pour HFT

Tableau 7 — Comparatif VPS pour IC Markets cTrader

Tuning du runtime Tokio

Par défaut, Tokio démarre un thread worker par cœur logique. Pour un moteur HFT, on veut un contrôle fin : 1 thread dédié au réseau, 1 thread dédié à la stratégie, et les cœurs restants libres pour le système. On utilise pour cela un `Runtime` configuré manuellement :

RUST · SRC/MAIN.RS — HFT RUNTIME

```
use tokio::runtime::{Builder, Runtime};
use core_affinity;
pub fn build_hft_runtime(worker_threads: usize) -> Runtime {
    Builder::new_multi_thread()
        .worker_threads(worker_threads)
        .enable_all()
        .thread_name("hft-worker")
        // LIFO pour les tâches I/O : meilleure cache locality sur les bursts
        .io_lifo_slot_capacity(64)
        // Bloom filter pour le dispatch des événements
        .event_interval_budget(64, 64)
        // Pas de park : le runtime reste actif même sans travail
        // (critique pour ne pas ajouter de latence à la première tick)
        .disable_lifo_slot()
        .build()
        .expect("failed to build HFT runtime")
}
// Pinning du thread réseau sur cœur 0, stratégie sur cœur 1
pub fn pin_threads() {
    let cores = core_affinity::get_core_ids().expect("core_affinity");
    // Tâche réseau (réception des ticks) sur cœur 0
    core_affinity::set_for_current(cores[0]);
    // Pour les tâches enfants, il faut utiliser spawn_blocking avec affinité
}
```

Files lock-free : crossbeam-queue

Pour la communication entre la tâche réseau (productrice de ticks) et la tâche stratégie (consommatrice), il faut une file **lock-free** au lieu d'un canal MPSC Tokio standard qui utilise un verrou interne. La crate `crossbeam-queue` fournit `ArrayQueue`, une file bornée *lock-free* basée sur le pattern *bounded MPMC* de Dmitry Vyukov :

RUST · SRC/PIPE.RS

```
use crossbeam_queue::ArrayQueue;
use std::sync::Arc;
pub struct TickPipe {
    queue: Arc<ArrayQueue<Quote>>,
```

```
}
impl TickPipe {
    pub fn new(capacity: usize) -> Self {
        Self { queue: Arc::new(ArrayQueue::new(capacity)) }
    }
    // Côté producteur (tâche réseau) : push non-bloquant
    // Si la file est pleine, le tick est droppé (latence > capacité de traitement)
    #[inline(always)]
    pub fn push(&self, q: Quote) -> bool {
        self.queue.push(q).is_ok()
    }
    // Côté consommateur (tâche stratégie) : busy-poll
    #[inline(always)]
    pub fn pop(&self) -> Option<Quote> {
        self.queue.pop().ok()
    }
}
// Usage typique : capacité 4096 (= ~2s de ticks à 2000/s)
// En dessous, on risque des drops si la stratégie a un spike de calcul
```

Zero-copy avec bytes::Bytes

Quand on reçoit un `ProtoMessage`, son payload est un `Vec`. Si on le clone à chaque passage entre tâches (network → parser → spot_mgr), on fait des allocations inutiles. La crate `bytes` fournit `Bytes`, une structure de référence comptée qui permet de **slicer sans copie**. Adapter le type de payload de `Vec` à `Bytes` dans vos wrappers de messages.

Désactivation de Nagle et tuning socket

Déjà vu au chapitre 5 mais à répéter car crucial : `TCP_NODELAY` doit être activé immédiatement après le `connect()`. Sans cela, les petites écritures (typiquement < 100 octets pour un heartbeat) sont bufferisées jusqu'à 40 ms par l'algorithme de Nagle, ajoutant une latence aléatoire qui ruine l'arbitrage. À cela s'ajoute le *SO_KEEPALIVE* pour détecter les connexions mortes côté OS, et le *SO_RCVBUF*/*SO_SNDBUF* à augmenter à 256 KB pour absorber les bursts de ticks.

Mesure de latence : hdrhist

Pour optimiser, il faut mesurer. La crate `hdrhist` implémente les *HDR Histograms* qui capturent les percentiles p50, p95, p99 et p999 sans overhead notable. Instrumentez chaque étape :

RUST · SRC/METRICS.RS

```
use hdrhist::Histogram;
use std::time::Instant;
static LATENCY_HIST: once_cell::sync::Lazy<
    std::sync::Mutex<Histogram<u64>>
> = once_cell::sync::Lazy::new(|| {
    std::sync::Mutex::new(Histogram::new_with_bounds(1, 10_000_000, 3).unwrap())
});
pub fn record_latency(start: Instant, label: &str) {
    let elapsed_us = start.elapsed().as_micros() as u64;
    LATENCY_HIST.lock().unwrap().record(elapsed_us).unwrap();
    if elapsed_us > 5000 {  // > 5 ms = alarme
        tracing::warn!(label, elapsed_us, "latency spike");
    }
}
// Périodiquement (toutes les 60s), dump percentiles
pub fn dump_latency_stats() {
```

```
    let h = LATENCY_HIST.lock().unwrap();
    tracing::info!(
        p50_us = h.value_at_quantile(0.5),
        p95_us = h.value_at_quantile(0.95),
        p99_us = h.value_at_quantile(0.99),
        p999_us = h.value_at_quantile(0.999),
        max_us = h.max(),
        "latency stats (µs)"
    );
}
```

CHAPITRE 13 · RÉSILIENCE

Gestion d'erreurs et résilience

Un moteur d'arbitrage en production passe plus de temps à gérer les cas d'erreur qu'à exécuter des opportunités. Ce chapitre couvre les patterns indispensables : reconnexion avec backoff exponentiel, circuit breaker, mapping des codes d'erreur ProtoOAErrorCode vers des actions concrètes, et slippage guard.

Reconnexion avec backoff exponentiel

RUST · SRC/ERROR.RS

```rust
use std::time::Duration;
use tokio::time::sleep;
pub struct ReconnectStrategy {
    initial_delay: Duration,
    max_delay: Duration,
    attempt: u32,
}
impl ReconnectStrategy {
    pub fn new() -> Self {
        Self {
            initial_delay: Duration::from_secs(1),
            max_delay: Duration::from_secs(60),
            attempt: 0,
        }
    }
    pub async fn next_delay(&mut self) -> Duration {
        // Backoff exponentiel avec jitter : delay = min(max, initial * 2^attempt) × (0.5 + random)
        let base = self.initial_delay.as_millis() as u64;
        let exp = base.saturating_mul(2u64.saturating_pow(self.attempt));
        let capped = exp.min(self.max_delay.as_millis() as u64);
        let jitter = (capped as f64) * (0.5 + rand::random::<f64>() * 0.5);
        self.attempt = self.attempt.saturating_add(1);
        let delay = Duration::from_millis(jitter as u64);
        tracing::warn!(attempt = self.attempt, delay_ms = delay.as_millis(),
            "reconnect attempt");
        sleep(delay).await;
        delay
    }
    pub fn reset(&mut self) {
        self.attempt = 0;
    }
}
// Circuit breaker : si > 5 reconnects en 1 min → pause 5 min
pub struct CircuitBreaker {
    failure_count: std::sync::atomic::AtomicU32,
    last_reset: std::sync::atomic::AtomicI64,
}
impl CircuitBreaker {
    pub fn should_trip(&self) -> bool {
        let now = chrono::Utc::now().timestamp();
        let last = self.last_reset.load(std::sync::atomic::Ordering::Relaxed);
        if now - last > 60 {
            self.failure_count.store(0, std::sync::atomic::Ordering::Relaxed);
            self.last_reset.store(now, std::sync::atomic::Ordering::Relaxed);
        }
        let count = self.failure_count.fetch_add(1,
            std::sync::atomic::Ordering::Relaxed);
        if count >= 5 {
            tracing::error!("circuit breaker triggered → pause 5 min");
            return true;
        }
        false
    }
}
```

Mapping ProtoOAErrorCode → actions

L'énumération `ProtoOAErrorCode` définit ~50 codes d'erreur métier. Voici la stratégie recommandée pour les plus critiques :

Code	Constante	Action recommandée
1	OA_AUTH_TOKEN_EXPIRED	Refresh token via ProtoOARefreshTokenReq (2173), puis re-account_auth
2	ACCOUNT_NOT_AUTHORIZED	Re-account_auth immédiat
67	CONNECTIONS_LIMIT_EXCEEDED	Backoff 60s, puis retry ; alerter si persiste
108	REQUEST_FREQUENCY_EXCEEDED	Throttle les requêtes, doubler l'intervalle minimum
114	SYMBOL_NOT_FOUND	Re-fetch_symbols ; ignorer ce symbole
117	NO_QUOTES	Pas de prix : attendre, ne pas order
118	NOT_ENOUGH_MONEY	Réduire volume de 50%, alerter opérateur
119	MAX_EXPOSURE_REACHED	Stop nouveaux ordres, clôturer positions existantes
125	TRADING_BAD_VOLUME	Vérifier le lot step ; log critique
126	TRADING_BAD_STOPS	SL/TP trop proches : recalculer
131	PENDING_EXECUTION	Ordre en cours : attendre ExecutionEvent, ne pas re-soumettre
132	TRADING_DISABLED	Marché fermé ou compte suspendu : arrêt trading
134	UNABLE_TO_CANCEL_ORDER	Ordre déjà fill ou expiré : reconcile
136	SHORT_SELLING_NOT_ALLOWED	Inverser direction ou abandonner opportunité

Tableau 8 — Codes d'erreur ProtoOAErrorCode critiques

Slippage guard

Le *slippage* est la différence entre le prix attendu au moment de l'envoi de l'ordre et le prix effectif du *fill*. En marché calme sur IC Markets, il est typiquement de 0.1-0.3 pips. Si le moteur observe un slippage supérieur à **2 pips** sur une jambe, cela indique soit une volatilité anormale, soit une latence excessive du moteur lui-même. Dans les deux cas, il faut désactiver temporairement la stratégie (60s) pour éviter une cascade de pertes :

RUST · SRC/ERROR.RS — SLIPPAGE GUARD

```
pub struct SlippageGuard {
    threshold_pips: f64,
    cooldown_until: std::sync::atomic::AtomicI64,
}
impl SlippageGuard {
    pub fn new(threshold_pips: f64) -> Self {
        Self {
            threshold_pips,
            cooldown_until: std::sync::atomic::AtomicI64::new(0),
        }
    }
    pub fn check(&self, expected_price: f64, filled_price: f64,
                 decimals: u8) -> bool
    {
        if self.is_in_cooldown() {
            return false;  // trading désactivé
        }
        let pip = 10f64.powi(-(decimals as i32 - 1));
        let slippage_pips = (filled_price - expected_price).abs() / pip;
        if slippage_pips > self.threshold_pips {
            tracing::error!(
                expected = expected_price,
```

```
                filled = filled_price,
                slippage_pips,
                "slippage critique → cooldown 60s"
            );
            let until = chrono::Utc::now().timestamp() + 60;
            self.cooldown_until.store(until,
                std::sync::atomic::Ordering::Relaxed);
            return false;
        }
        true
    }
    fn is_in_cooldown(&self) -> bool {
        let until = self.cooldown_until.load(std::sync::atomic::Ordering::Relaxed);
        chrono::Utc::now().timestamp() < until
    }
}
```

Logging structuré avec rotation

En production, le moteur doit écrire des logs structurés JSON avec rotation quotidienne, pour permettre une analyse post-mortem en cas d'incident. La crate `tracing-appender` gère la rotation :

RUST · SRC/MAIN.RS — TRACING

```
use tracing_subscriber::{fmt, prelude::*, EnvFilter};
use tracing_appender::rolling;
pub fn init_tracing(log_dir: &str) -> tracing_appender::non_blocking::WorkerGuard {
    let file_appender = rolling::daily(log_dir, "utilustrader.log");
    let (non_blocking, guard) = tracing_appender::non_blocking(file_appender);
    tracing_subscriber::registry()
        .with(EnvFilter::from_env("RUST_LOG"))
        .with(fmt::layer().with_writer(non_blocking).json())
        .with(fmt::layer().with_writer(std::io::stderr))  // aussi stderr pour systemd
        .init();
    guard  // Garder vivant pour la durée du programme
}
```

CHAPITRE 14 · BACKTESTING

Mode démo, backtesting et replay de ticks

Avant toute mise en production réelle, le moteur doit être validé sur un compte de démonstration IC Markets, puis en backtesting sur des données historiques rejouées. Ce chapitre décrit l'architecture d'un mode *paper trading*, d'un *replay engine*, et la stratégie de tests d'intégration.

Paper Trading Interceptor

Le *paper trading* consiste à intercepter les ordres avant leur envoi au serveur, et à simuler localement leur exécution au prix *mid* (`(bid + ask) / 2`) avec un slippage estimé. Cela permet de valider la logique de détection et le PnL théorique sans risquer de capital réel.

RUST · SRC/PAPER.RS

```
use crate::orders::{OrderManager, OrderResult, TradeSide};
use crate::spot::SpotManager;
pub struct PaperTradingInterceptor {
    spot_mgr: SpotManager,
    positions: dashmap::DashMap<i64, PaperPosition>,
```

```
    next_order_id: std::sync::atomic::AtomicI64,
    slippage_pips: f64,
}
#[derive(Debug, Clone)]
pub struct PaperPosition {
    pub symbol_id: i64,
    pub side: TradeSide,
    pub volume: i64,
    pub entry_price: f64,
    pub opened_at_ms: u64,
}
impl PaperTradingInterceptor {
    pub fn new(spot_mgr: SpotManager, slippage_pips: f64) -> Self {
        Self {
            spot_mgr, slippage_pips,
            positions: dashmap::DashMap::new(),
            next_order_id: std::sync::atomic::AtomicI64::new(1),
        }
    }
    pub async fn send_market_order(
        &self,
        symbol_id: i64,
        side: TradeSide,
        volume: i64,
        decimals: u8,
    ) -> OrderResult {
        let q = match self.spot_mgr.get(symbol_id) {
            Some(q) => q,
            None => return OrderResult::Rejected {
                code: 117, reason: "NO_QUOTES".into(),
            },
        };
        let mid = (int_to_price(q.bid, decimals)
                 + int_to_price(q.ask, decimals)) / 2.0;
        // Slippage simulé : unfavorable au trader
        let pip = 10f64.powi(-(decimals as i32 - 1));
        let fill_price = match side {
            TradeSide::BUY  => mid + self.slippage_pips * pip,
            TradeSide::SELL => mid - self.slippage_pips * pip,
        };
        let order_id = self.next_order_id.fetch_add(1,
            std::sync::atomic::Ordering::Relaxed);
        self.positions.insert(order_id, PaperPosition {
            symbol_id, side, volume,
            entry_price: fill_price,
            opened_at_ms: q.timestamp_ms,
        });
        tracing::info!(
            order_id, symbol_id, side = ?side,
            fill_price, "PAPER: ordre simulé");
        OrderResult::Filled { order_id, volume, price: fill_price }
    }
}
```

Replay Engine : rejouer des ticks enregistrés

Pour backtester, on enregistre d'abord les `ProtoOASpotEvent` reçus pendant une session live (typiquement une journée de trading EUR/USD), puis on les rejoue à la même vitesse (ou en accéléré) via une source `QuoteStream` qui implémente le même trait que le SpotManager live.

RUST · SRC/REPLAY.RS

```
use std::fs::File;
use std::io::{BufReader, Read};
```

```
// Format binaire compact : [u64 ts_ms][i64 symbol_id][i64 bid][i64 ask]
// 32 octets par tick → 1 jour de ticks à 2000/s = ~5 GB
const TICK_RECORD_SIZE: usize = 32;
pub struct ReplaySource {
    reader: BufReader<File>,
    speed_multiplier: f64,
    start_ts: Option<u64>,
    real_start: Option<std::time::Instant>,
}
impl ReplaySource {
    pub fn open(path: &str, speed: f64) -> std::io::Result<Self> {
        let f = File::open(path)?;
        Ok(Self {
            reader: BufReader::new(f),
            speed_multiplier: speed,
            start_ts: None,
            real_start: None,
        })
    }
    pub async fn next_quote(&mut self) -> Option<Quote> {
        let mut buf = [0u8; TICK_RECORD_SIZE];
        self.reader.read_exact(&mut buf).ok()?;
        let ts_ms = u64::from_be_bytes(buf[0..8].try_into().unwrap());
        let sym = i64::from_be_bytes(buf[8..16].try_into().unwrap());
        let bid = i64::from_be_bytes(buf[16..24].try_into().unwrap());
        let ask = i64::from_be_bytes(buf[24..32].try_into().unwrap());
        // Respecter la timing réelle (ou accéléré)
        if let Some(start) = self.start_ts {
            let real_start = self.real_start.unwrap();
            let elapsed_real = std::time::Instant::now().duration_since(real_start);
            let elapsed_replay_ms = (ts_ms - start) as f64
                / self.speed_multiplier;
            let expected = std::time::Duration::from_millis(elapsed_replay_ms as u64);
            if elapsed_real < expected {
                tokio::time::sleep(expected - elapsed_real).await;
            }
        } else {
            self.start_ts = Some(ts_ms);
            self.real_start = Some(std::time::Instant::now());
        }
        Some(Quote {
            symbol_id: sym, bid, ask, timestamp_ms: ts_ms,
        })
    }
}
// Enregistrement côté live
pub fn record_tick(path: &str, q: &Quote) -> std::io::Result<()> {
    use std::io::Write;
    let mut f = std::fs::OpenOptions::new()
        .append(true).create(true).open(path)?;
    let mut buf = [0u8; TICK_RECORD_SIZE];
    buf[0..8].copy_from_slice(&q.timestamp_ms.to_be_bytes());
    buf[8..16].copy_from_slice(&q.symbol_id.to_be_bytes());
    buf[16..24].copy_from_slice(&q.bid.to_be_bytes());
    buf[24..32].copy_from_slice(&q.ask.to_be_bytes());
    f.write_all(&buf)?;
    Ok(())
}
```

Métriques de backtesting

Sur une session backtestée, le moteur doit calculer les métriques standard de performance :

Métrique	Formule	Seuil acceptable
Win rate	Trades gagnants / total trades	> 55%
Profit factor	Somme gains / somme pertes	> 1.5
Max Drawdown	Plus forte baisse du capital cumulé	< 10%
Sharpe ratio	(Rendement - risk-free) / volatilité	> 1.5
Average slippage	Moyenne |fill - expected|	< 0.5 pip
Latency p99	99e percentile latence détection→envoi	< 5 ms
Opportunités/jour	Compteur brut	Variable selon marché
Captured %	Opportunités capturées / détectées	> 60%

Tableau 9 — Métriques de backtesting

CHAPITRE 15 · MISE EN PRODUCTION

Checklist de mise en production

Avant tout déploiement sur un compte réel IC Markets, valider chacun des points suivants. Un seul point manquant peut coûter cher — typiquement, une perte maximale admise de 1% du capital par trade devient 10% si le slippage guard est absent, ou 100% si l'authentification n'est pas rafraîchie automatiquement.

Sécurité et configuration

- **Secrets en variables d'environnement** : clientId, clientSecret, accessToken ne doivent JAMAIS apparaître dans le code source, les logs, ou les fichiers de config commités. Utiliser `std::env::var()` ou un gestionnaire de secrets (Vault, AWS Secrets Manager).
- **VPS LD5** : provisionné, accessible en SSH par clé uniquement (pas de mot de passe), firewall limité au port 5035 en sortie et 22 en entrée.
- **Certificats TLS à jour** : rustls utilise webpki-roots qui se met à jour avec chaque release de la crate. Vérifier `cargo update` mensuellement.
- **NTP synchronisé** : la précision des timestamps cTrader dépend de l'horloge système. Installer `chrony` et vérifier `chronyc tracking` montre un offset < 5 ms.

Test et validation

- **Compte démo IC Markets** : faire tourner le moteur pendant au moins **5 jours ouvrés complets** sur démo avant tout passage en réel. Surveiller le PnL, les messages d'erreur, et les latences p99.
- **Tests d'intégration** : mock TCP server local qui répond aux auth et envoie des spots pré-enregistrés. Vérifier le bon comportement sur reconnect, heartbeat manqué, ordre reject.
- **Backtesting** : rejouer au moins 30 jours de ticks EURUSD/USDCHF/EURCHF. Validater que le win rate et le profit factor sont acceptables.

Monitoring et alerting

• **Prometheus + Grafana** : exposer les métriques (latence p99, PnL cumulé, nombre d'opportunités, taux d'échec) sur `/metrics` via `prometheus_exporter`.

• **Alerting** : alerter si latence p99 > 50 ms, si PnL < -2% sur 1h, si heartbeat watchdog se déclenche, ou si circuit breaker activé.

• **Rotation des logs** : `logrotate` quotidien avec rétention 30 jours, et `coredump` configuré (`ulimit -c unlimited`) pour analyse post-crash.

Risque et capital

• **Sizing du capital** : ne jamais risquer plus de **1% du capital** par trade. Si capital = 10 000 €, max loss par trade = 100 €, soit ~10 pips sur 1 lot EURUSD.

• **Circuit breaker hardware** : onduleur (UPS) pour absorber les coupures courtes, et 4G backup en cas de panne fibre LD5.

> **ATTENTION · LIMITES DE L'ARBITRAGE RETAIL**
> L'arbitrage triangulaire retail est en concurrence directe avec des institutions (HFT firms, banks) qui exécutent le même cycle en moins de 100 µs avec du matériel colocalisé et du FPGA. Sur IC Markets, les opportunités profitables *après coûts* sont rares (souvent < 10/jour en marché calme) et généralement capturées avant qu'un client retail ne puisse y accéder. Ce document décrit l'implémentation techniquement correcte, mais ne garantit en aucun cas la rentabilité. **Toujours commencer en démo**, avec un capital que vous pouvez perdre intégralement.

ANNEXE A · RÉFÉRENCE

Référence des PayloadType (2100–2188)

Liste exhaustive des 89 PayloadType de l'énumération `ProtoOAPayloadType` tels que définis dans `spotware.rs`. Les types en **gras** sont ceux utilisés par un moteur d'arbitrage (voir chapitre 3).

ID	Nom Protobuf	Type	ID	Nom Protobuf	Type
2100	**PROTO_OA_APPLICATION_AUTH_REQ**	REQ	2145	PROTO_OA_GET_TICKDATA_REQ	REQ
2101	**PROTO_OA_APPLICATION_AUTH_RES**	RES	2146	PROTO_OA_GET_TICKDATA_RES	RES
2102	**PROTO_OA_ACCOUNT_AUTH_REQ**	REQ	2147	PROTO_OA_ACCOUNTS_TOKEN_INVALIDATED_EVENT	EVT
2103	**PROTO_OA_ACCOUNT_AUTH_RES**	RES	2148	PROTO_OA_CLIENT_DISCONNECT_EVENT	EVT
2104	PROTO_OA_VERSION_REQ	REQ	2149	PROTO_OA_GET_ACCOUNTS_BY_ACCESS_TOKEN_REQ	REQ
2105	PROTO_OA_VERSION_RES	RES	2150	PROTO_OA_GET_ACCOUNTS_BY_ACCESS_TOKEN_RES	RES
2106	**PROTO_OA_NEW_ORDER_REQ**	REQ	2151	PROTO_OA_GET_CTID_PROFILE_BY_TOKEN_REQ	REQ
2107	PROTO_OA_TRAILING_SL_CHANGED_EVENT	EVT	2152	PROTO_OA_GET_CTID_PROFILE_BY_TOKEN_RES	RES
2108	**PROTO_OA_CANCEL_ORDER_REQ**	REQ	2153	PROTO_OA_ASSET_CLASS_LIST_REQ	REQ
2109	PROTO_OA_AMEND_ORDER_REQ	REQ	2154	PROTO_OA_ASSET_CLASS_LIST_RES	RES

ID	Nom Protobuf	Type	ID	Nom Protobuf	Type
2110	PROTO_OA_AMEND_POSITION_SLTP_REQ	REQ	2155	**PROTO_OA_DEPTH_EVENT**	EVT
2111	**PROTO_OA_CLOSE_POSITION_REQ**	REQ	2156	**PROTO_OA_SUBSCRIBE_DEPTH_QUOTES_REQ**	REQ
2112	PROTO_OA_ASSET_LIST_REQ	REQ	2157	PROTO_OA_SUBSCRIBE_DEPTH_QUOTES_RES	RES
2113	PROTO_OA_ASSET_LIST_RES	RES	2158	PROTO_OA_UNSUBSCRIBE_DEPTH_QUOTES_REQ	REQ
2114	**PROTO_OA_SYMBOLS_LIST_REQ**	REQ	2159	PROTO_OA_UNSUBSCRIBE_DEPTH_QUOTES_RES	RES
2115	**PROTO_OA_SYMBOLS_LIST_RES**	RES	2160	PROTO_OA_SYMBOL_CATEGORY_REQ	REQ
2116	PROTO_OA_SYMBOL_BY_ID_REQ	REQ	2161	PROTO_OA_SYMBOL_CATEGORY_RES	RES
2117	PROTO_OA_SYMBOL_BY_ID_RES	RES	2162	PROTO_OA_ACCOUNT_LOGOUT_REQ	REQ
2118	PROTO_OA_SYMBOLS_FOR_CONVERSION_REQ	REQ	2163	PROTO_OA_ACCOUNT_LOGOUT_RES	RES
2119	PROTO_OA_SYMBOLS_FOR_CONVERSION_RES	RES	2164	PROTO_OA_ACCOUNT_DISCONNECT_EVENT	EVT
2120	PROTO_OA_SYMBOL_CHANGED_EVENT	EVT	2165	PROTO_OA_SUBSCRIBE_LIVE_TRENDBAR_RES	RES
2121	PROTO_OA_TRADER_REQ	REQ	2166	PROTO_OA_UNSUBSCRIBE_LIVE_TRENDBAR_RES	RES
2122	PROTO_OA_TRADER_RES	RES	2167	PROTO_OA_MARGIN_CALL_LIST_REQ	REQ
2123	PROTO_OA_TRADER_UPDATE_EVENT	EVT	2168	PROTO_OA_MARGIN_CALL_LIST_RES	RES
2124	PROTO_OA_RECONCILE_REQ	REQ	2169	PROTO_OA_MARGIN_CALL_UPDATE_REQ	REQ
2125	PROTO_OA_RECONCILE_RES	RES	2170	PROTO_OA_MARGIN_CALL_UPDATE_RES	RES
2126	**PROTO_OA_EXECUTION_EVENT**	EVT	2171	PROTO_OA_MARGIN_CALL_UPDATE_EVENT	EVT
2127	**PROTO_OA_SUBSCRIBE_SPOTS_REQ**	REQ	2172	PROTO_OA_MARGIN_CALL_TRIGGER_EVENT	EVT
2128	**PROTO_OA_SUBSCRIBE_SPOTS_RES**	RES	2173	PROTO_OA_REFRESH_TOKEN_REQ	REQ
2129	PROTO_OA_UNSUBSCRIBE_SPOTS_REQ	REQ	2174	PROTO_OA_REFRESH_TOKEN_RES	RES
2130	PROTO_OA_UNSUBSCRIBE_SPOTS_RES	RES	2175	PROTO_OA_ORDER_LIST_REQ	REQ
2131	**PROTO_OA_SPOT_EVENT**	EVT	2176	PROTO_OA_ORDER_LIST_RES	RES
2132	**PROTO_OA_ORDER_ERROR_EVENT**	EVT	2177	PROTO_OA_GET_DYNAMIC_LEVERAGE_REQ	REQ
2133	PROTO_OA_DEAL_LIST_REQ	REQ	2178	PROTO_OA_GET_DYNAMIC_LEVERAGE_RES	RES
2134	PROTO_OA_DEAL_LIST_RES	RES	2179	PROTO_OA_DEAL_LIST_BY_POSITION_ID_REQ	REQ
2135	PROTO_OA_SUBSCRIBE_LIVE_TRENDBAR_REQ	REQ	2180	PROTO_OA_DEAL_LIST_BY_POSITION_ID_RES	RES
2136	PROTO_OA_UNSUBSCRIBE_LIVE_TRENDBAR_REQ	REQ	2181	PROTO_OA_ORDER_DETAILS_REQ	REQ
2137	PROTO_OA_GET_TRENDBARS_REQ	REQ	2182	PROTO_OA_ORDER_DETAILS_RES	RES

ID	Nom Protobuf	Type	ID	Nom Protobuf	Type
2138	PROTO_OA_GET_TRENDBARS_RES	RES	2183	PROTO_OA_ORDER_LIST_BY_POSITION_ID_REQ	REQ
2139	PROTO_OA_EXPECTED_MARGIN_REQ	REQ	2184	PROTO_OA_ORDER_LIST_BY_POSITION_ID_RES	RES
2140	PROTO_OA_EXPECTED_MARGIN_RES	RES	2185	PROTO_OA_DEAL_OFFSET_LIST_REQ	REQ
2141	PROTO_OA_MARGIN_CHANGED_EVENT	EVT	2186	PROTO_OA_DEAL_OFFSET_LIST_RES	RES
2142	**PROTO_OA_ERROR_RES**	RES	2187	PROTO_OA_GET_POSITION_UNREALIZED_PNL_REQ	REQ
2143	PROTO_OA_CASH_FLOW_HISTORY_LIST_REQ	REQ	2188	PROTO_OA_GET_POSITION_UNREALIZED_PNL_RES	RES
2144	PROTO_OA_CASH_FLOW_HISTORY_LIST_RES	RES			

Tableau 10 — Liste exhaustive des 89 PayloadType ProtoOAPayloadType

Références et ressources externes

Ressource	URL
Dépôt source (Raul Gherman)	github.com/raul-gherman/ctrader-openapi-rust
Documentation officielle Open API	help.ctrader.com/open-api/
Forum cTrader (communauté)	community.ctrader.com/forum/open-api
Portail développeur (création app)	openapi.ctrader.com/
Equinix LD5 (colocalisation)	www.equinix.com/data-centers/europe-colocation/united-kingdom/london-data-centers
IC Markets cTrader (overview)	www.icmarkets.com/forex-trading-platforms/ctrader
Quacky crate (sérialisation)	crates.io/crates/quacky
Quack-builder (génération proto)	crates.io/crates/quack-builder
Tokio (runtime async)	tokio.rs
rustls (TLS pur Rust)	github.com/rustls/rustls

Tableau 11 — Ressources externes

Document généré pour le projet utilustrader.io — adaptation du dépôt ctrader-openapi-rust à un moteur d'arbitrage Rust sur IC Markets cTrader Open API v2. Version 1.0 — Novembre 2026.